

PDF → JSON Project — Master Guide

Purpose: This document consolidates the complete project context, decisions, working code, step-by-step workflow, testing process, normal usage, and instructor explanation developed during the project conversation. It is intended as a reference for continuing the project in a future chat without having to reconstruct the project history.

Important: This guide records the current project state. If an actual run on the full real PDF produces an unexpected result, the actual output/error should take priority and the parser should be investigated rather than assumed correct.

1. Project Goal

The project reads a structured PDF and extracts only the required information into structured JSON.

```
PDF
↓
PyMuPDF
↓
Text extraction
↓
Procedure separation
↓
Regex extraction
↓
Structured Python data
↓
output.json
```

Required extracted information: Procedure, Time, Dur, STEP ID, and status grouped as success and failure.

Ignored information: Mode, Source, Priority, STEP description, and table TIME.

2. Actual PDF Data Structure

The tested PDF contains a metadata block approximately like:

```
id="374ixx"
id="wesft5"
Procedure : abbnm          Time:31-dec-2025 11:44:13.103          Mode:Manual
Source: abbvdskh          Dur:203.96(s)          Priority: Low
```

Then a table approximately like:

```
id="3qzm35"
id="ywpitm"
STEP ID          STEP          TIME          STATUS
01.0001          hdhggjhgdhj          11:44:20          Success
01.0002          dchxc khjgjh          22:22:30          Failure
```

The status is exactly Success or Failure. STEP IDs such as 01.0001 must remain strings.

3. Desired JSON Output

```
[
  {
    "Procedure": "abbnm",
    "Time": "31-dec-2025 11:44:13.103",
    "Dur": "203.96(s)",
    "status": {
      "success": ["01.0001"],
      "failure": ["01.0002"]
    }
  }
]
```

```

    }
}
]

```

For the full PDF, the JSON is a list containing one object for each procedure.

4. Final Project Structure

```

samarth/
├── main.py
├── pdf_parser.py
├── input/
│   └── input.pdf
├── output.json
└── PyMuPDF-1.18.18

```

For the current simple workflow, no archive folder, utils.py, json_generator.py, or other extra Python files are required.

5. Why There Are Two Python Files

main.py: controls the overall workflow: opens the PDF, reads all pages, separates procedures, calls the parser functions, collects the results, and generates JSON.

pdf_parser.py: contains the extraction functions. `extract_process_info()` extracts Procedure, Time, and Dur. `extract_status()` extracts STEP IDs and groups them by Success/Failure.

A separate `json_generator.py` was discussed earlier, but it is not necessary because JSON generation is only a few lines and is kept in `main.py`. A separate `utils.py` is also unnecessary for the current scope because there are not enough shared helper functions to justify it.

6. Environment

Working environment established during the project:

```

Python 3.6.5
PyMuPDF (fitz) 1.18.18, 32-bit
Older PyCharm on the college system
Project folder: samarth

```

The project does not use AI, LLMs, OCR, or machine learning for the actual extraction. Normal Python, PyMuPDF, regular expressions, and JSON are used.

7. Final pdf_parser.py

```

import re

def extract_process_info(text):
    Procedure = ""
    Time = ""
    Dur = ""

    match = re.search(r"Procedure\s*:\s*(.*?)\s*(?=\s*Time\s*:\s*|$)", text, re.IGNORECASE | re.MULTILINE)
    if match:
        Procedure = match.group(1).strip()

    match = re.search(r"Time\s*:\s*(.*?)\s*(?=\s*Dur\s*:\s*|$)", text, re.IGNORECASE | re.MULTILINE)
    if match:
        Time = match.group(1).strip()

    match = re.search(r"Dur\s*:\s*(.*?)\s*(?=\s*Priority\s*:\s*|$)", text, re.IGNORECASE | re.MULTILINE)
    if match:
        Dur = match.group(1).strip()

```

```

    return Procedure, Time, Dur

def extract_status(text):
    status_data = {}
    lines = text.splitlines()

    for line in lines:
        match = re.match(r"^s*(\d+\.\d+)\s+.*?\b(Success|Failure)\s*$", line, re.IGNORECASE)

        if match:
            STEP_ID = match.group(1)
            STATUS = match.group(2).lower()

            if STATUS not in status_data:
                status_data[STATUS] = []

            status_data[STATUS].append(STEP_ID)

    return status_data

```

8. Final main.py

```

import fitz
import re
import json

from pdf_parser import extract_process_info, extract_status

pdf = fitz.open("input/input.pdf")

all_text = ""

for page in pdf:
    all_text = all_text + page.get_text() + "\n"

procedure_blocks = re.split(r"(?=Procedure\s*:)", all_text, flags=re.IGNORECASE)
procedure_blocks = [block.strip() for block in procedure_blocks if block.strip()]

all_processes = []

for block in procedure_blocks:
    Procedure, Time, Dur = extract_process_info(block)
    status_data = extract_status(block)

    process_data = {
        "Procedure": Procedure,
        "Time": Time,
        "Dur": Dur,
        "status": status_data
    }

    all_processes.append(process_data)

with open("output.json", "w") as file:
    json.dump(all_processes, file, indent=4)

print("Total procedures:", len(all_processes))
print("JSON file created successfully.")

pdf.close()

```

9. Steps 1–5 — Initial Build

The early steps established the Python/PyMuPDF environment, confirmed that fitz could be imported, opened the PDF, read PDF text, and extracted one procedure successfully.

The first-page test produced the expected structure:

```
{
  'Procedure': 'abbnm',
  'Time': '31-dec-2025 11:44:13.103',
  'Dur': '203.96(s)',
  'status': {
    'success': ['01.0001'],
    'failure': ['01.0002']
  }
}
```

10. Step 6 — Read All PDF Pages

```
import fitz

pdf = fitz.open("input/input.pdf")

all_text = ""

for page in pdf:
    all_text = all_text + page.get_text() + "\n"

print("Total pages:", len(pdf))
print("Total text length:", len(all_text))

pdf.close()
```

Success criteria: total pages should be greater than zero and total text length should be greater than zero. The user reported Step 6 completed.

11. Step 7 — Count Procedures

```
import fitz
import re

pdf = fitz.open("input/input.pdf")

all_text = ""

for page in pdf:
    all_text = all_text + page.get_text() + "\n"

procedures = re.findall(r"Procedure\s*:", all_text, re.IGNORECASE)

print("Total pages:", len(pdf))
print("Total procedures found:", len(procedures))

pdf.close()
```

The procedure count becomes a checkpoint for later steps. The user reported Step 7 completed.

12. Step 8 — Separate Procedure Blocks

```
import fitz
import re

pdf = fitz.open("input/input.pdf")

all_text = ""
```

```

for page in pdf:
    all_text = all_text + page.get_text() + "\n"

procedure_blocks = re.split(r"(?=Procedure\s*:)", all_text, flags=re.IGNORECASE)

procedure_blocks = [block.strip() for block in procedure_blocks if block.strip()]

print("Total pages:", len(pdf))
print("Total procedures:", len(procedure_blocks))

for i, block in enumerate(procedure_blocks, start=1):
    print("Procedure", i)
    print(block[:200])
    print("-----")

pdf.close()

```

The count should agree with Step 7. The user reported Step 8 completed.

13. Step 9 — Create One Dictionary per Procedure

```

import fitz
import re

from pdf_parser import extract_process_info, extract_status

pdf = fitz.open("input/input.pdf")

all_text = ""

for page in pdf:
    all_text = all_text + page.get_text() + "\n"

procedure_blocks = re.split(r"(?=Procedure\s*:)", all_text, flags=re.IGNORECASE)
procedure_blocks = [block.strip() for block in procedure_blocks if block.strip()]

all_processes = []

for block in procedure_blocks:
    Procedure, Time, Dur = extract_process_info(block)
    status_data = extract_status(block)

    process_data = {
        "Procedure": Procedure,
        "Time": Time,
        "Dur": Dur,
        "status": status_data
    }

    all_processes.append(process_data)

print("Total procedures:", len(all_processes))

for process in all_processes:
    print(process)

pdf.close()

```

The user reported Step 9 as completed/assumed correct.

14. Step 10 — Create output.json

```

import fitz
import re

```

```

import json

from pdf_parser import extract_process_info, extract_status

pdf = fitz.open("input/input.pdf")

all_text = ""

for page in pdf:
    all_text = all_text + page.get_text() + "\n"

procedure_blocks = re.split(r"(?=Procedure\s*:", all_text, flags=re.IGNORECASE)
procedure_blocks = [block.strip() for block in procedure_blocks if block.strip()]

all_processes = []

for block in procedure_blocks:
    Procedure, Time, Dur = extract_process_info(block)
    status_data = extract_status(block)

    process_data = {
        "Procedure": Procedure,
        "Time": Time,
        "Dur": Dur,
        "status": status_data
    }

    all_processes.append(process_data)

with open("output.json", "w") as file:
    json.dump(all_processes, file, indent=4)

print("Total procedures:", len(all_processes))
print("JSON file created successfully.")

pdf.close()

```

output.json is created automatically. It does not need to be manually created first. If it already exists, write mode replaces it.

15. Step 11 — Verify output.json

Open output.json and compare it with the original PDF. Check Procedure, Time, Dur, Success/Failure STEP IDs, procedure count, and several procedures. Ensure unwanted fields such as Mode, Source, Priority, STEP description, and table TIME are absent.

STEP IDs must remain strings such as 01.0001. If the JSON is wrong, do not manually edit it to hide the problem; investigate the parser.

16. Step 12 — Edge-Case Testing

The purpose was to check that the parser is not working only for one exact example.

Relevant tests included: multiple Success STEP IDs, multiple Failure STEP IDs, mixed Success/Failure, only Success, only Failure, spacing variations, STEP IDs remaining strings, multiple procedures, procedures across page boundaries, and empty/unusual sections.

If a test fails, record the expected versus actual output and investigate the parser rather than manually correcting output.json.

Do not replace the real input PDF with test data unless it is backed up.

17. Step 13 — Final Clean Project

After verification/testing, temporary testing code and random test files are removed. Keep the final `main.py`, `pdf_parser.py`, `input/input.pdf`, `output.json`, and the required PyMuPDF/environment files.

Do not add unnecessary files merely to make the project look larger. The current two-file Python architecture is intentional.

18. Step 14 — Normal Usage With One PDF

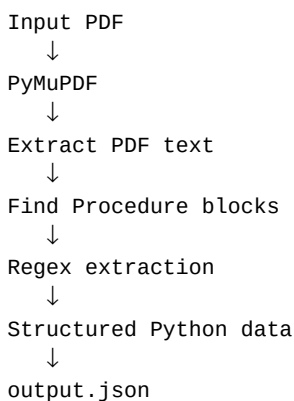
For now, use only one input PDF and one output JSON. No archive system is required.

1. Get the PDF.
2. Put it in `input/`.
3. Rename it to `input.pdf`.
4. Run `main.py`.
5. Open `output.json`.
6. Check the extracted data.

The old input PDF is replaced when a new one is processed. If the old PDF may be needed later, copy it somewhere else before replacing it. The old `output.json` is automatically overwritten by the next run.

The program currently expects a compatible PDF structure matching the established format. A completely different PDF format may require parser changes.

19. Step 15 — Final Project Workflow



The project intentionally extracts only the required fields. PyMuPDF reads the PDF; regular expressions identify known text patterns; Python dictionaries organize the data; JSON stores the final structured result.

20. Instructor Explanation

Short project explanation: Our project takes a structured PDF as input and extracts the required process information from it. We use PyMuPDF to read the PDF and extract its text. Then we use Python and regular expressions to identify the Procedure, Time, Duration, STEP IDs, and their Success or Failure status. The extracted information is organized into Python dictionaries and finally stored as structured JSON in `output.json`. We ignore fields that are not required, such as Mode, Source, Priority, STEP description, and the table Time.

21. Common Instructor Questions

Why PyMuPDF?

PyMuPDF is used to open the PDF and extract its text so Python can process the required information.

Why regular expressions?

Regular expressions identify specific patterns such as Procedure, Time, Dur, STEP IDs, Success, and Failure.

Why JSON?

JSON provides a structured, machine-readable representation of the information extracted from the PDF.

Why two Python files?

main.py controls the overall workflow, while pdf_parser.py contains the extraction logic. This keeps the code organized and easier to maintain.

Why not AI?

The PDF has a known structure, so deterministic parsing with Python, PyMuPDF, and regular expressions is sufficient and predictable for this task.

Why are STEP IDs strings?

STEP IDs are identifiers rather than mathematical values. Keeping them as strings preserves their exact format, including leading zeros.

What happens with a new PDF?

Replace input/input.pdf with the new compatible PDF, run main.py, and a new output.json is generated.

22. Important Project Limitations

The current parser is designed for the PDF structure that was tested. It should not be described as a universal PDF parser. If a new PDF has materially different labels, layout, or STEP-row formatting, the parser may require adjustment.

The project does not claim universal bug detection, OCR, AI interpretation, or support for arbitrary PDF formats.

23. What To Do If Something Goes Wrong

Always stop at the first checkpoint that fails.

Example:

Step 7 says 20 procedures

Step 10 says 17 procedures

→ Stop.

→ Do not continue.

→ Do not manually edit output.json.

→ Check the extraction/separation logic.

If Python displays an error, preserve the complete error message. If Python runs but the output is wrong, preserve the input PDF and output.json and inspect the actual extracted data.

24. Normal Usage vs Development

Development: Steps 6–12 were used to read the full PDF, count procedures, separate blocks, build dictionaries, generate JSON, verify results, and test edge cases.

Normal use: Put a compatible PDF in input/input.pdf, run main.py, and inspect output.json. You do not repeat all development/testing steps for every PDF.

25. 9 AM–4 PM Completion Plan

A 7-hour workday is a reasonable target for carrying out Steps 6 onward if the environment and current code remain working. A practical schedule is:

9:00–9:30 Reopen project and check environment

9:30–10:15 Step 6

10:15–10:45 Step 7

10:45-11:30 Step 8
11:30-12:15 Step 9
12:15-1:00 Break/lunch
1:00-1:45 Step 10
1:45-2:30 Step 11
2:30-3:15 Step 12
3:15-3:40 Step 13
3:40-4:00 Steps 14-15 review/final run

This is a target, not a guarantee. If a checkpoint fails, stop and resolve it rather than rushing forward.

26. Final Checklist

- Python/PyMuPDF environment works
- Actual PDF can be opened
- All PDF pages are read
- Procedure count is known
- Procedure blocks are separated
- Each procedure becomes a dictionary
- output.json is generated
- output.json matches the PDF
- Success/Failure STEP IDs are correct
- STEP IDs remain strings
- Unwanted fields are absent
- Edge cases have been tested
- Temporary testing files are cleaned up
- Normal new-PDF workflow is understood
- Instructor explanation is understood

27. Current Status

The core PDF → JSON project workflow is considered complete based on the steps developed in the conversation. The next practical action is to carry out the steps on the college system and verify the full real PDF. If the actual full dataset exposes a formatting variation, that observed result should be used to refine the parser.

28. One-Sentence Project Summary

The project extracts selected structured process and STEP-status information from a compatible PDF using PyMuPDF and Python regular expressions, then converts the extracted information into structured JSON.

Master reference generated from the project workflow and decisions established in this conversation.